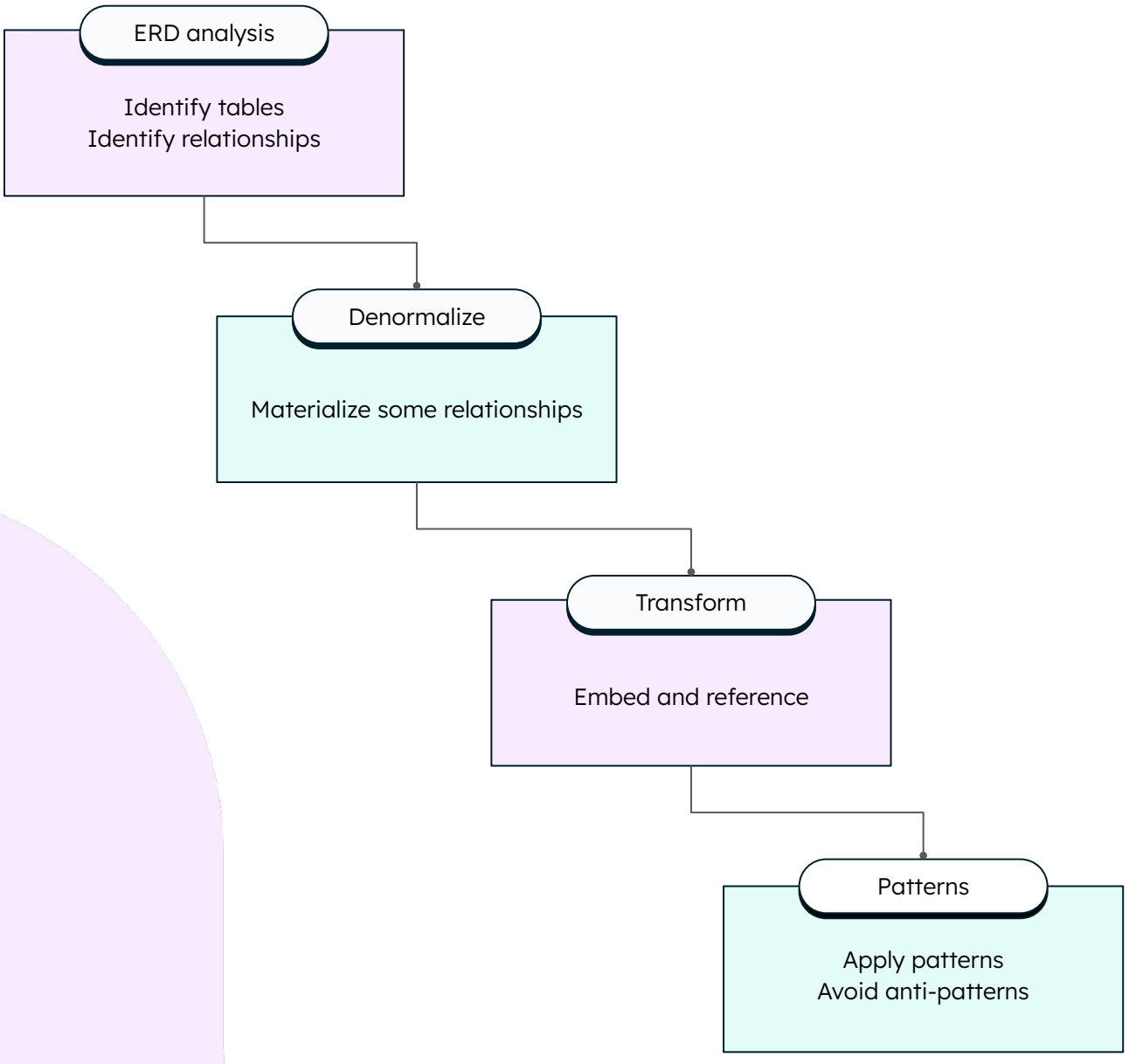
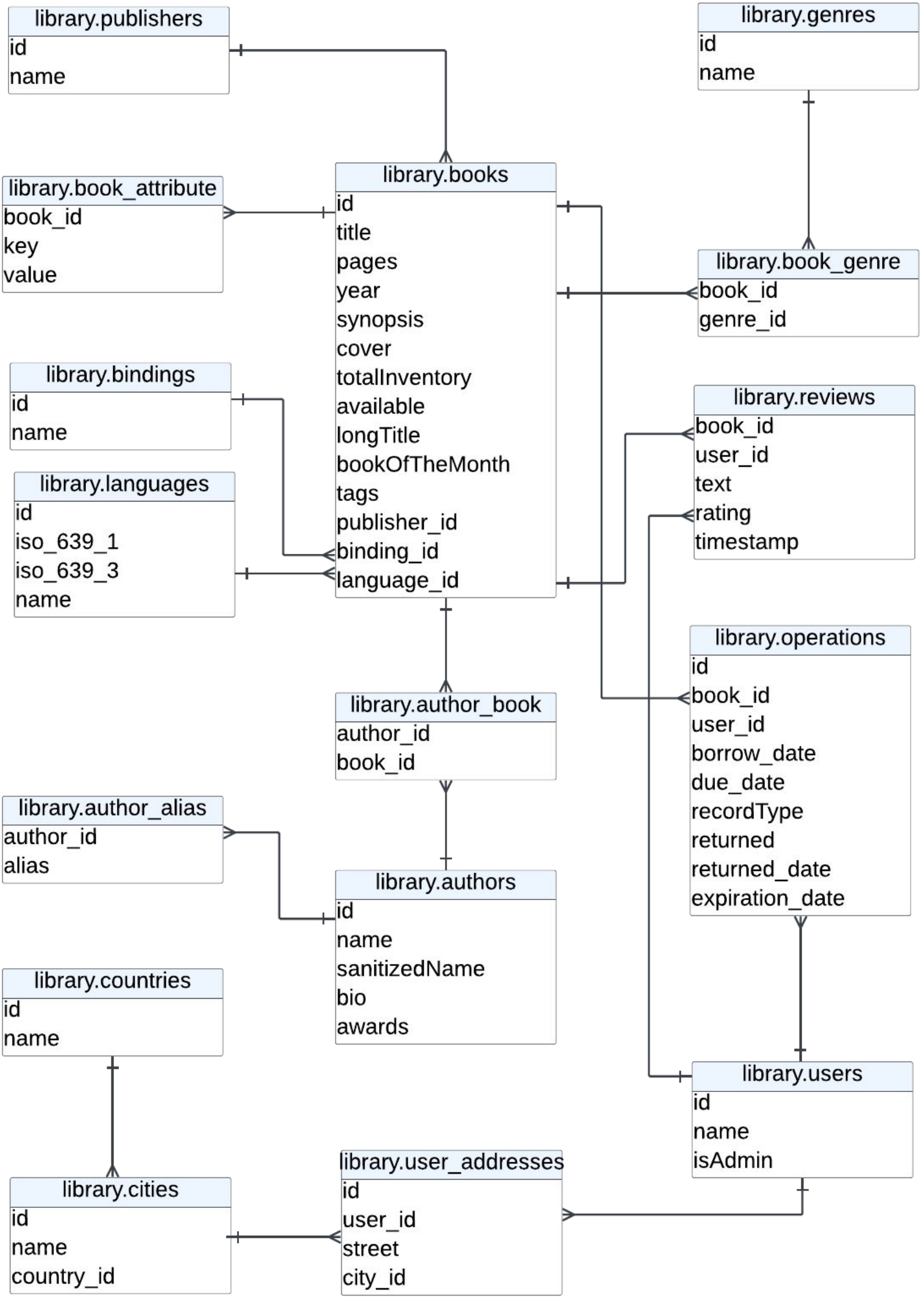
Data Modeling Methodology from RDBMS to MongoDB




library.publishers
id
name
library.genres
id
name
library.books
id
title
pages
year
synopsis
cover
totalInventory
available
longTitle
bookOfTheMonth
tags
publisher_id
binding_id
language_id
library.book_attribute
book_id
key
value
library.book_genre
book_id
genre_id
library.bindings
id
name
library.languages
id
iso_639_1
iso_639_3
name
library.reviews
book_id
user_id
text
rating
timestamp
library.operations
id
book_id
user_id
borrow_date
due_date
recordType
returned
returned_date
expiration_date
library.author_book
author_id
book_id
library.author_alias
author_id
alias
library.authors
id
name
sanitizedName
bio
awards
library.countries
id
name
library.users
id
name
isAdmin
library.cities
id
name
country_id
library.user_addresses
id
user_id
street
city_id

Functional Requirements

The system should allow **readers** to:

- R1. Browse a catalogue of library books.
- R2. View the extended details of a book — including title, author, and synopsis — and reviews from other readers.
- R3. Make a book reservation if the book is available. Book reservations expire after 12 hours.
- R4. See a list of their unexpired reservations.
- R5. See a list of all books they have borrowed from the library and their due dates.
- R6. View the details for an author, including all of the books they have published.

The system should allow **librarians** to:

- L1. See a list of all book reservations, including their expiration time.
- L2. See a list of all borrowed books, including their due date.
- L3. Filter reservations and borrowed books by user.
- L4. Lend books to readers.
- L5. Mark books as returned.

Library Data

- A. The library has **10k active readers weekly**.
- B. The library has about **250k books**.
- C. Readers will consult the library website on a **weekly basis** and borrow books **every week**.
- D. Most readers **reserve** and **borrow** up to **3 books** every week.
- E. **Reservations** can be made online, but books can only be **borrowed** on-site.
- F. Readers rarely read more than **5 book reviews**.
- G. Readers browse about **25 books** before reserving.
- H. Readers leave **reviews for 1 in 5 books** they read.

Performance Requirements

- P1. The book details page that shows book availability and recent reviews should load in less than **300ms**.
- P2. Readers should be able to reserve a book in less than **3 seconds**.
- P3. Librarians should be able to lend a book or mark a book as returned in less than **3 seconds**.
- P4. Readers should be able to see the list with all of their reservations and borrowed books in less than **500ms**.

Identify tables

Tables

Classify tables by their type: Entity, Lookup, Associative, or Weak

Table Name	Type
	Entity
	Entity
	Entity
	Entity
	Entity
	Entity
	Entity
	Entity
	Entity
	Entity
	Entity
	Entity
	Entity

Worksheet Page 1: *Tables*

Table types

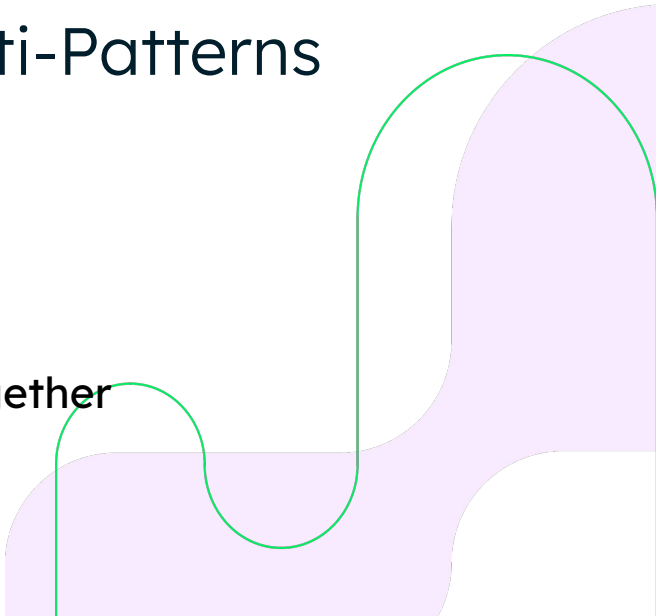
- Entity: strong business entities, could potentially exist alone.
- Lookup: dimension values for attributes in other tables
- Associative: contain references to resolve many-to-many relationships
- Weak: many-to-one data subordinate to a parent table

Embedding vs. Referencing

Guideline Name	Question	Embed	Reference
Simplicity	Would keeping the pieces of information together lead to a simpler data model and code?	Yes	
Go Together	Do the pieces of information have a "has-a," "contains," or similar relationship?	Yes	
Query Atomicity	Does the application query the pieces of information together?	Yes	
Update Complexity	Are the pieces of information updated together?	Yes	
Archival	Should the pieces of information be archived at the same time?	Yes	
Cardinality	Is there a high cardinality (current or growing) in a "many" side of the relationship?	No	Yes
Data Duplication	Would data duplication be too complicated to manage and undesired?	No	Yes
Document Size	Would the combined size of the pieces of information take too much memory or transfer bandwidth for the application?	No	Yes
Document Growth	Would the embedded piece grow without bound?	No	Yes
Workload	Are the pieces of information written at different times in a write-heavy workload?		Yes
Individuality	For the children side of the relationship, can the pieces exist by themselves without a parent?		Yes

Schema Design Anti-Patterns

1. Massive arrays
2. Massive number of collections
3. Unnecessary indexes
4. Bloated documents
5. Separating data that is accessed together



From Relational to Document Model

Data Modeling for MongoDB

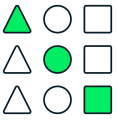
With MongoDB, you can design the most optimal schema for your use case instead of a generic schema for any use case.

Core MongoDB Design Concepts

Embedding

You can store documents within other documents to represent relationships and complex hierarchies.

Embedding allows for storing related data within a single document, unlike relational databases that use separate tables and JOINs to represent relationships.



Arrays

The value of a field in a document can be an array of values, which can themselves be documents.

Arrays enable many data modeling options, particularly for one-to-many and many-to-many relationships, allowing you to store related data together.

Polymorphism

You can store documents with different fields within the same collection.

Polymorphism allows similar types of data to be stored in the same collection and retrieved together.

Polymorphism also allows you to evolve your schema without downtime. Different versions of the schema can coexist.

When modeling data for MongoDB, your workload should drive your data model.

Schema Validation

Highly recommended for maintaining data integrity

Defined at the collection level

Enforced by the database regardless of the connection source

```
{
   bsonType: 'object',
   required: ['name', 'biography'],
   properties: {
       name: { bsonType: 'string' },
       biography: { bsonType: 'string' },
       aliases: {
           bsonType: 'array',
           maxItems: 10
       }
   }
};
```

```
{
   bsonType: 'object',
   required: [
     'rating',
     'reviewer',
     'book'
   ],
   properties: {
     text: { bsonType: 'string' },
     rating: {
       bsonType: 'int',
       minimum: 1,
       maximum: 5
     },
     reviewer: { bsonType: 'objectId' },
     book: { bsonType: 'objectId' },
   }
};
```

Modeling Relationships

One-to-one. Embed in the natural parent entity.
Exception: Avoid embedding if it causes bloated documents, leading to high memory or bandwidth usage.

One-to-few. Embed on the "one" side.
Exceptions:
- If the “few” are frequently accessed on their own.
- Avoid embedding if it causes bloated documents, leading to high memory or bandwidth usage.

One-to-many (medium cardinality). Follow the “Embedding vs referencing guidelines” sheet.
Most importantly, store together if accessed together.

One-to-zillions (unbounded growth). Use references on the "many" side.

Many-to-many. Reference on the more often queried side.
Exception: If the array with references could grow unbounded, reference from the other side instead.

MongoDB.

Schema Design Patterns and Anti-Patterns

Data Modeling for MongoDB

Use schema design patterns to optimize your data model based on how your application queries and uses data. Avoid anti-patterns that can degrade performance.

Patterns

Extended reference

An "in-between" solution between embedding and referencing for minimizing the number of joins for frequent operations

Use it when data is stored in separate collections that need to be joined frequently

Computed

Precompute and store values, offsetting the computation on write instead of read

Addresses the case where computational tasks are performed frequently on data retrieval

Use it on high read-to-write ratio on documents where computation is needed

Inheritance

Simplify the retrieval of similar data by storing it in a polymorphous collection and using a field to distinguish between the types

Addresses the issue with documents that are similar but still have distinct features; allows simpler retrieval of data as it can coexist

Anti-Patterns

Unbounded arrays

Large growing arrays with an unlimited number of elements

Use referencing on the “many” side or embed only a subset of the array

Bloated documents

Oversized documents containing data that isn’t accessed together

They inflate the working set, potentially exceeding the WiredTiger internal cache and degrading performance

“MongoDB design patterns are reusable solutions for many of the commonly occurring use cases encountered when designing applications that leverage persistence in MongoDB.”

Daniel Coupal
Senior Staff Developer Advocate, MongoDB

Learn more about MongoDB

MongoDB for Learners

MongoDB provides you with free resources to gain product knowledge, practice skills with hands-on labs, and get certified.

Earn free skill badges to validate and showcase your skills.

https://mdb.link/university

MongoDB Articles & Tutorials

Elevate your development skills and efficiency with MongoDB articles and tutorials. Check out our content on Medium to discover insights about using MongoDB. Dive into the technical details in our tutorials on DEV.

https://medium.com/mongodb

https://dev.to/mongodb

Join the MongoDB Community

Learn from community members around the world, attend local MongoDB events, and be inspired.

Explore our community.

https://mdb.link/mdb-community